

Balloon Popping Competition

David Amorim
Cordeiro

*Faculty of Engineering
University of Porto
Porto, Portugal
up202108820@up.pt*

Ema Maria Monteiro
Martins

*Faculty of Engineering
University of Porto
Porto, Portugal
up202402794@up.pt*

Isabel Maria Couto da
Silva

*Faculty of Engineering
University of Porto
Porto, Portugal
up201904925@up.pt*

Pedro Miguel Meruge
Ferreira

*Faculty of Engineering
University of Porto
Porto, Portugal
up202409828@up.pt*

Abstract—We present the development and training of a competitive multi-agent reinforcement learning system for autonomous robot teams in a dynamic battle arena. The system employs the POCA training algorithm combined with self-play and curriculum learning to train agents capable of strategic positioning, opponent engagement, and resource management. Using Battle Royale mechanics, wind forces and obstacles, the agents learned to cooperative and competitive behaviors. The performance evaluation shows the system’s teams coordination, decision-making and responses to a dynamic environment, using win-rates, Elo ratings and game-specific metrics.

Index Terms—Robotics, Multi-Agent Reinforcement Learning, Simulation, Object Detection

I. INTRODUCTION

The field of Multi-Agent Reinforcement Learning (MARL) has witnessed significant advances recently, driving the development of autonomous systems capable of operating in complex, dynamic environments. Unlike single-agent systems, which only require agents to navigate their physical surroundings, MARL systems must also account for the non-stationary behavior of other co-adapting entities. A fundamental challenge in this domain is the alignment of individual actions with collective team goals, particularly in competitive scenarios where resource acquisition is time-constrained. In this context, simulated competitive environments serve as a critical testbed for developing robust strategic multi-agent systems.

This paper explores the emergence of interesting strategies within a simulated competitive “balloon-popping” environment. In this task, teams of robotic agents must coordinate their movements to locate and pursue resource targets (balloons) within a predefined temporal window, while simultaneously defending against aggressive opponents. The environment presents a high-dimensional observation space where agents must balance offensive maneuvers, intended to maximize their team’s score, with defensive positioning and zone navigation to hinder the opposition.

To investigate these dynamics, we utilize the Unity ML-Agents Toolkit, leveraging its physics-based simulation capabilities to create a challenging training ground featuring shrinking safe zones and curriculum-based difficulty progression. The agents are trained using the Posthumous Credit Assignment (POCA) algorithm, employing a Centralized Training with Decentralized Execution (CTDE) framework. This

approach utilizes a centralized critic during training to estimate group value, while the resulting policy allows each agent to make independent, local decisions during execution. This architecture enables the emergence of complex global team objectives from local observations, reflecting real-world constraints found in domains such as swarm robotics.

II. STATE OF THE ART

Unity is a well-known framework to create games. It is widely used because of its easiest for learning how the framework works and the support it has from other users.

Unity ML-Agents is an open-source framework that bridges Unity simulations with machine learning by enabling agents to learn through Reinforcement Learning and Imitation Learning [6]. Operating as integrated GameObject components, agents navigate environments by processing sensory observations and executing actions to maximize feedback-driven rewards. The framework facilitates a seamless transition between rule-based heuristics and neural network inference, providing a robust foundation for multi-agent systems.

Multi-agent reinforcement learning (MARL) addresses the challenge of developing agents that learn to coordinate or compete toward shared or individual goals. A key paradigm is Centralized Training for Decentralized Execution (CTDE), which enables agents to leverage centralized information during training while executing independently with only local observations during deployment [4]. This approach proves scalable and effective without requiring communication during execution. Multi-agent competition has also proven to be a powerful learning mechanism. Through competitive interaction and adaptive curricula, agents spontaneously develop coordination strategies and tool use capabilities [2], demonstrating the potential of multi-agent systems to achieve complex, human-relevant behaviors that surpass other self-supervised approaches.

III. ARCHITECTURE

This chapter begins by defining the specific rules and mechanics of the competitive balloon-popping scenario designed for our experiments. It then formalizes the environment by detailing the observation and action spaces, followed by an explanation of the training architecture, which utilizes a Centralized Training with Decentralized Execution (CTDE)

framework. Subsequently, the chapter outlines the reward signals and physical modeling choices implemented to shape agent behavior. Finally, it concludes with a discussion on the requirements for recreating this work in alternative simulators or real-world robotics settings.

A. Description of the competition

In the base version of our competition, two opposing teams of agents fight within a closed square arena to finish the match with the highest number of balloons. Matches take a total of forty five seconds to complete.

Each agent begins with three balloons attached to its body, and can pick up new ones, up to a maximum of five. Two small squared zones spawn a balloon that can be picked up, and replenishes it after a small interval. Agents can pop the balloons attached to the bodies of other agents, by slamming a “spike”, attached to their front, against them. Agents also begin each match with a random position and orientation. Finally, agents can choose to activate a boost state and become faster for 2 seconds. The boost has a cooldown of 5 seconds.

Agents that lose all their balloons are not eliminated; they remain active in the match and can continue to attack opponents or gather new balloons. Conversely, if an agent reaches the maximum capacity of five balloons, it can no longer consume health pickups. These balloons remain in the arena, available for other agents to collect.

The arena contains a circular border that shrinks towards the center as the match progresses, emulating a battle royale game. Agents that fall outside this border are pushed by a constant force towards the center of the arena. The arena has several walls placed inside it in a non-symmetric pattern, to make navigation and gameplay more varied.

If both teams have an equal number of balloons at the end of a round, the result ends in a draw, otherwise the team with the highest number of balloons are the winners.

For an image of the game environment, refer to Figure 1.

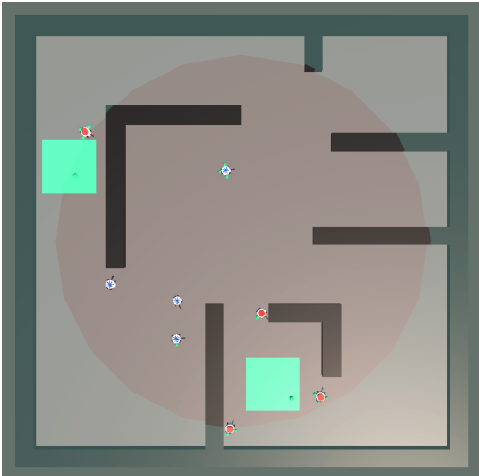


Fig. 1: Final arena with the balloon spawning areas and the robots.

B. Observation Space

The agents’ observation space is composed of intrinsic and extrinsic values. The intrinsic measures are those captured by the agent itself (those that would be captured by the internal sensors of the robot, in a robotics scenario), and the extrinsic measures are those transmitted to them by an external system, which is detailed in section III-D. The intrinsic observations are: linear and angular velocity, boost availability, boost in use and the number of active balloons. Besides these, there is also the hit detection and distance information provided by 2 (actually we use four in total, two in each layer, since Unity enforces a limit of 50 rays per sensor, and we wanted an higher resolution, but they function as a singular unit) Unity ML-Agents RayPerceptorSensor3D, simulating a Lidar (unlike a true Lidar, the rays cast by the sensor are only planar). The sensors are composed of 100 rays, spread across a plane with a 180 degree aperture, positioned at the front of each agent, one set higher up to detect an agent’s second-layer (described in section III-F), and the other positioned more towards the bottom, giving information on other robots’ body shape and balloons. As for extrinsic observations, we have: the position in the arena, the orientation (given as the normalized forward vector), the team ID (represented as a one-hot encoded vector of size two), the availability and direction towards the balloons of the spawn zones (given as a 2d vector), and a factor of round progression, used as a way to track the progression of the shrinking border.

C. Action Space

The action space of the agents is made up of two continuous actions (linear and angular velocities), and one discrete action (enabling or disabling the boost).

D. Training

Our training was based on the concept of Centralized Training with Decentralized Execution (CTDE). This framework has become quite popular for MARL scenarios, and has seen some success with well-known projects, such as the Open AI Hide and Seek project [2]. The idea is to decouple the critic and the actors. The centralization happens with the critic, and only during training. It aggregates and uses privileged information, to get a good-enough understanding on the global state of the environment, and better estimate the value needed for training. During execution, each actor relies only on the trained policy, and their local observations, and does not possess access to the privileged information that the critic had in training. To achieve this in Unity, we made use of the ML-Agents MA-POCA algorithm which aggregates the local observations of every agent during training, to obtain a global estimate. For this to work, we needed to record every observation within the agents’ own function, even those which are extrinsic. In other settings, this would be captured by an entirely different process and passed directly to the critic, but in order to comply with the ML-Agents expectations and conceptually capture the architecture, there was no better solution we found of implementing it.

Regarding the specific configuration of the agent’s neural network and training hyperparameters, we defined a model with 2 hidden layers of 512 units each. The decision to use a larger network size (512 units) was driven by the need to capture complex geometric features from the ray perceptions, allowing the agents to potentially distinguish between team structures based on shape patterns. The training was set to run for a total of 15 million steps, using a batch size of 2048, a buffer size of 20480, and a learning rate of 0.0003.

To facilitate the learning of complex behaviors, we implemented a Curriculum Learning strategy divided into three distinct phases. In the first phase (0 to 1 million steps), agents are spawned in a restricted area near the center of the arena without wind or obstacles, allowing them to focus on basic movement and close-quarters interaction. The spawn area gradually expands until it covers the full arena. In the second phase (1 million to 3 million steps), the "Battle Royale" wind mechanic is introduced, ramping up from 0 to a maximum force of 40, teaching the agents to resist the center-pushing force while maneuvering. Finally, in the third phase (3 million to 15 million steps), physical obstacles are enabled, presenting the agents with the full complexity of the environment, requiring them to navigate walls while managing combat and wind forces.

E. Reward Signals

Win and loss rewards were attributed to the team as a whole, and used together with individual rewards and penalties, attributed to each agent, to try and better shape behaviour. The rewards used and their values are described in Table I. Step penalty was initially used to encourage the agents to be more aggressive, by attributing a small negative reward at every step of an episode. This was later found to be redundant, as explained in the next chapter. The wall-hit penalty was given to encourage the agents to better navigate the map without colliding with the central walls. It is capped so that its total strength never overpowers more important signals. The reward for popping a balloon was set to double the penalty of losing one balloon, encouraging more aggressive behaviour. The reward for picking up a new balloon was also set to a smaller value to try to encourage a similar aggressive effect.

TABLE I: Agent Reward Function Mapping

Event / Outcome	Reward Value
Match Victory (Win)	+1.0
Match Defeat (Loss)	-1.0
Draw	0.0
Pop an opponent’s balloon	+0.2
Pop a teammate’s balloon	-0.1
Loose a balloon	-0.1
Collect a balloon (Spawn area)	+0.075
Step Penalty	Disabled
Wall Hit Penalty	-0.0001 (Max -0.05 per episode)

F. Modelling

The agents were modeled as cylindrical shapes, a choice motivated by the resemblance with several real-world robots

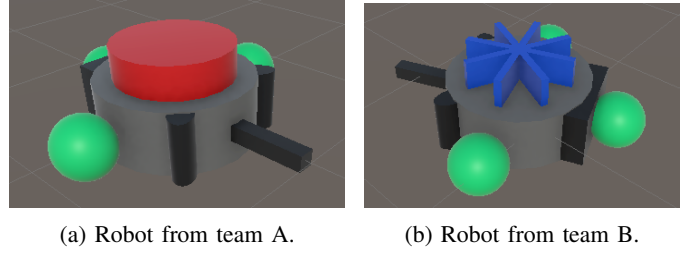


Fig. 2: Robots with distinct top layers.

with similar movement. We added other geometry around the body to make it more distinct to the sensors at different angles, and easier to infer other robots’ orientation. The spike is set at the front, and was modelled as a rectangular prism. We added a second layer, on top of each agent’s cylinder. Its purpose was to provide some geometric shape, which the upper sensor could use to learn to distinguish both teams. For that effect, we aimed for shapes which were as distinct from each other, as well as from a flat surface, due to the arena walls, as possible. We ended up modelling a smaller smooth cylinder for the red team, and an 8-pointed star for the blue team (Fig. 2). Additionally, the balloons are modeled as simple spheres placed around the robot.

The arena was also modelled using only Unity’s default cube primitive. To aid in visualization during training, after the match ends, the floor flashes with the color of the victorious team (or stays grey, in case of a draw).

G. Recreation of our work

The choice to use Unity came from the maturity the ML-Agents package has gained since its introduction for reinforcement learning projects, and due to our familiarity with the game engine. However, recreating this scenario in a robotics-specific simulator, could be very advantageous, especially when considering trying to make it more realistic. The specific algorithm used for reinforcement learning is not in itself a requirement, as long as it is suitable and functions as centralized training with decentralized execution. In order to achieve this, a separate process is required. This process is responsible for capturing the extrinsic observations (such as the position in the arena) for each agent and transmitting it to them during both training and execution. For training specifically, this process would act as the privileged critic of the CTDE scenario, which joins all of the local observations from the agents (or other global observations one might decide to add), to better evaluate the value of the current global state. The training must, in the end, result in a policy that relies only upon the local observations of each agent. Considering a real-world scenario, one can easily see how a top-down vision system, for example, processed by a server, can be used to collect and transmit these observations.

Some mechanics of the competition are more easily translatable to actual robotics. The cylindrical shape we used for the agents, for example, was specifically chosen due to its similarity with differentially-wheeled robots. Movement,

popping balloons, and knowledge of how many balloons a particular robot has are feasible to acquire. Other aspects would require more creativity or a different solution to adapt, namely the shrinking border force that pressures the robots toward the center of the arena, the ability to spawn balloons, and particularly the ability, pick up new balloons.

IV. DIFFICULTIES AND CHANGES DURING MODELING

A. Redefining Winning Criteria and Reward Shaping

In the initial approach, the game’s objective was to pop all opponent balloons, and an episode would only conclude once a team was entirely eliminated. This led to a common issue in Reinforcement Learning known as “lazy agent” behavior. The agents learned that staying passive and hiding was statistically safer than risking a loss, as the negative reward for losing (or being popped) outweighed the perceived benefit of an aggressive pursuit. To mitigate this, we transitioned to a fixed time-limited match (45 seconds). The winning criteria was redefined: the team possessing the highest number of balloons at the end of the timer wins. Additionally, the maximum balloon capacity (5) was set higher than the starting amount (3), incentivizing resource gathering rather than just survival.

Critically, we observed that standard “Existence Penalties” (or Step Penalties), often used to encourage speed, were counter-productive in this fixed-duration setting. They acted as a constant negative drift that discouraged exploration. We effectively removed this penalty to allow agents to exist without losing value. Furthermore, we recalibrated the combat rewards: the penalty for friendly fire was significantly reduced (from -0.5 to -0.15) to prevent the “fear of collision” that paralyzed agents in early training, while the reward for popping enemies was increased to ensure aggression was statistically profitable.

B. Environmental Forces and Centripetal Bias

To encourage interaction and combat between agents, we explored environmental changes to promote proximity within the center of the arena.

The first strategy involved an arena layout with a flat central zone and surrounding planes inclined upwards towards the map extremities. By utilizing physics materials and gravity, we intended to simulate the difficulty of climbing slopes while rewarding the speed gained when moving toward the center. However, this approach proved problematic for the agents’ perception; the Raycast sensors, which were configured to scan horizontally relative to the agent, provided inaccurate readings (either hitting the floor or the sky) when the agent’s pitch changed on the inclined surfaces. While adding complex oblique sensors was a potential solution, it was discarded as it would disproportionately increase the observation space and training complexity.

Our final approach was to implement a “Battle Royale” style shrinking border with a dynamic centripetal force. As the match progresses, a circular safe zone shrinks towards the center. Agents caught outside this border are subjected to a force that pushes them inward. Crucially, this force is not constant; it scales based on the agent’s distance from

the safe zone and the elapsed match time. This mechanic effectively simulates increasing wind resistance or a magnetic pull, forcing specific “end-game” confrontations in the center without introducing the perceptual noise caused by uneven terrain.

V. RESULTS AND DISCUSSION

To evaluate the performance and behavior of the agents, we analyzed the training progression using the TensorBoard visualization toolkit. While standard reinforcement learning metrics such as Cumulative Reward and Policy Loss provided a high-level view of the learning stability, they were insufficient to fully characterize the complex team dynamics and strategic shifts occurring in the arena.

Therefore, we implemented a custom statistics recorder to track specific game-play events. These metrics were logged at periodic intervals (every 10,000 steps) for both teams independently, allowing for a comparative analysis of their evolution. The key performance indicators analyzed include:

- **Match Victories:** The total number of wins achieved by each team, serving as the primary measure of competitive success.
- **Opponent Balloons Popped:** A direct measure of offensive engagement and combat accuracy.
- **Teammate Balloons Popped:** Used to evaluate the agents’ ability to distinguish targets and coordinate safely.
- **Health Items Collected:** Tracking the agents’ efficiency in resource gathering and map control.
- **Average End-Game Balloons:** The average number of balloons held by the team at the conclusion of each match, serving as a proxy for defensive survival skills.
- **Wall Collisions:** Monitoring navigation precision and obstacle avoidance capabilities.

A. Initial Training

In the first training session, we simplified the environment to focus on core game dynamics. The agents were subjected to a centripetal force pushing them toward the center of the arena and were able to interact with health pickers. Additionally, we removed all internal walls during this stage to prioritize the learning of movement and combat mechanics without the added complexity of obstacle avoidance.

The reward function utilized during the initial training phase is detailed in Table II

TABLE II: Agent Reward Function Mapping - Initial Training

Event / Outcome	Reward Value
Match Victory (Win)	+1.0
Match Defeat (Loss)	-1.0
Collect a balloon (Spawn area)	+0.075
Pop an opponent’s balloon	+0.1
Pop a teammate’s balloon	-0.5
Step Penalty	-0.00005 (Max -0.2 per episode)
Wall Hit Penalty	-0.0001 (Max -0.05 per episode)

Figure 3 demonstrates that during training, although the cumulative rewards remained **below zero**, the agents showed

consistent improvement over time, indicating an effective learning process.



Fig. 3: Cumulative rewards during the first training session.

During the training process, the agents did not clearly determine whether prioritizing health collection was an effective strategy. This conclusion is supported by Figure 4, which shows a high degree of variance (randomness) in the amount of health items picked, indicating no clear learned pattern.

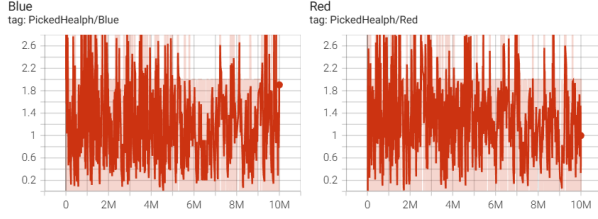


Fig. 4: Frequency of health items collected during the first training session.

Figure 5 demonstrates that the agents successfully learned to avoid popping teammates' balloons. The downward trend in this metric suggests that the penalty associated with friendly fire was effective in discouraging this behavior.

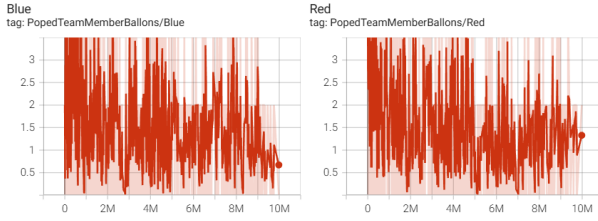


Fig. 5: Frequency of teammate balloons popped.

Based on Figures 6 and 7, it can be concluded that the agents prioritized defensive behaviors. This is evidenced by the increase in the average number of remaining balloons per episode, even as the number of opponent balloons popped decreased. Furthermore, Figure 8 shows a decrease in total eliminations for both teams, reinforcing the observation that the agents became more cautious and focused on survival rather than aggressive engagement.

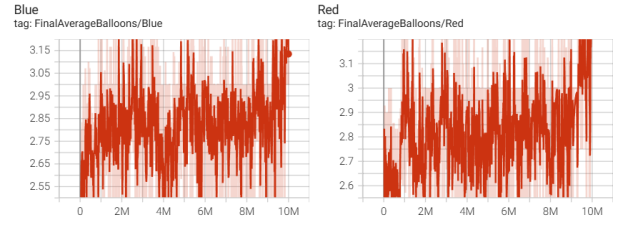


Fig. 6: Average number of balloons remaining at the end of each episode.

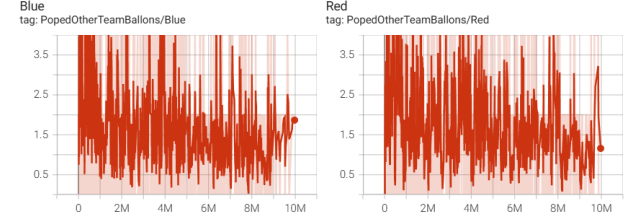


Fig. 7: Frequency of opponent balloons popped.

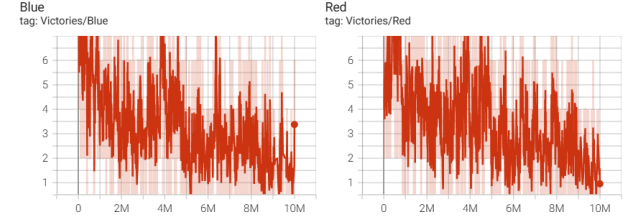


Fig. 8: Overall match outcomes and victory/defeat frequency.

The primary drawback of this initial approach was the emergence of excessively defensive behavior, which negatively impacted the agents' overall playability and engagement. The agents prioritized survival to avoid the loss penalty, resulting in a lack of offensive initiative. Consequently, these results led us to initiate a second training phase, where we adjusted specific reward values and refined the curriculum learning stages to encourage a better balance between defense and aggression.

B. Training with Improvements

Following the analysis of the initial results, specifically the passivity and lack of combat engagement, a second, more robust training phase was initiated. This phase introduced comprehensive changes to the reward structure and the curriculum learning schedule.

The setup for this training session incorporated a structured Curriculum Learning approach divided into three distinct phases to scaffold the agents' learning process:

- 1) Phase 1: Agents spawn in a restricted central area to force immediate interaction, with no wind or obstacles.
- 2) Phase 2: Wind mechanics are introduced and ramped up, teaching resistance to environmental forces.
- 3) Phase 3: Full arena complexity with obstacles enabled.

Regarding the reward function, critical adjustments were made to shift the optimal policy from "passive survival" to "active engagement." We removed the step penalty entirely, as it was counter-productive in a fixed-duration match (45s). We also tuned the combat incentives: the penalty for friendly fire was reduced (from -0.5 to -0.15) to prevent early-training paralysis, and the reward for popping enemies was doubled (to +0.2) to make aggression statistically more valuable than farming.

TABLE III: Agent Reward Function Mapping

Event / Outcome	Reward Value
Match Victory (Win)	+1.0
Match Defeat (Loss)	-1.0
Draw	0.0
Pop an opponent's balloon	+0.2
Pop a teammate's balloon	-0.1
Loose a balloon	-0.1
Collect a balloon (Spawn area)	+0.075
Step Penalty	Disabled
Wall Hit Penalty	-0.0001 (Max -0.05 per episode)

In the following figures, the red line represents the first training session, while the blue line refers to the second session. The second training session was longer as it employed the curriculum learning strategy that automatically adapted to different circumstances: starting without wind or walls, followed by augmenting the wind to maximum force, and finally incorporating all environmental constraints.

As shown in Figure 10, the cumulative reward, while not significantly different, is slightly higher with the new reward function.

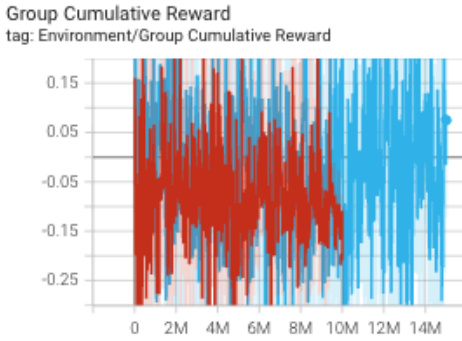


Fig. 9: Cumulative rewards during the first and the second training session.

Figure 11 shows that during the second training session, agents finished the episodes with fewer balloons compared to the first approach. This suggests a more developed attacking strategy and a significantly more competitive behavior.

Figure 12 also suggests an increase in the number of victories, driven by more competitive agent behavior. Although the number of losses also rose, there was a significant reduction in draws, resulting in an upward trend for the overall win rate.

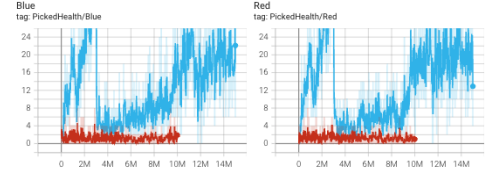


Fig. 10: Comparison of health item collection frequency between the initial training session (Red) and the improved curriculum training (Blue).

Figure 10 illustrates the frequency of health items collected per episode. The line for the initial training session maintained a consistently low and stagnant collection rate, indicating that agents failed to identify health pickups as a valuable resource. In contrast, the blue line (improved training) shows a dynamic learning curve. Initially, collection rates surge during the 'Bootcamp' phase (0-1M steps) as agents operate in a small, dense area. This is followed by a sharp drop at the onset of the 'Storm' phase (1M-4M steps), where the introduction of wind forces initially disrupts their navigation. However, the agents rapidly adapt, recovering and eventually surpassing previous performance peaks, demonstrating a robust mastery of resource gathering even under complex environmental constraints.

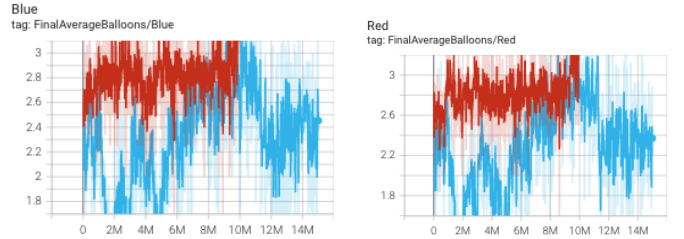


Fig. 11: Comparison of average remaining balloons per episode between the first and second train.

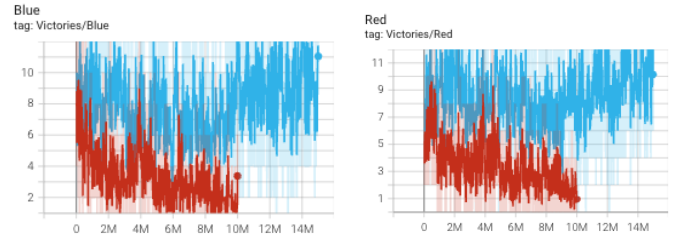


Fig. 12: Comparison of victories per episode between the first and second train.

Figure 13 shows that the agents did not fully learn to cooperate, as the frequency of popping teammates' balloons remained generally high. Although the rate initially began to decrease, it eventually reverted to higher levels. This could suggest that the agents attempted to employ strategies involving the sacrifice of a teammate to secure a win. However, this hypothesis was discarded after observing that the total number of victories did not increase accordingly.

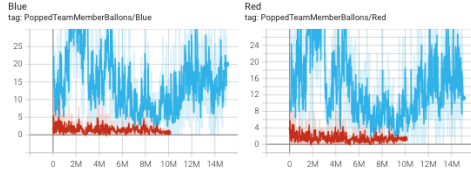


Fig. 13: Total instances of balloons popped by teammates during Training 1 and Training 2.

Figure 14 indicates a general decrease in wall collisions. However, some agents still get stuck in loops against walls. We believe that involving higher penalties for collisions or longer training sessions could resolve these sub-optimal behaviors.

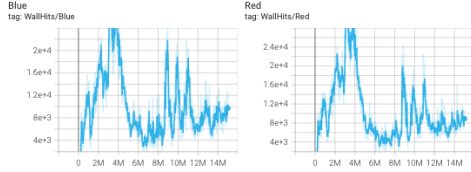


Fig. 14: Average number of wall collisions per episode during training.

C. Teams of four

As an extra goal, we explored larger team sizes by scaling up to teams of four robots. We first applied the existing training developed for two-robot teams, which performed well in this larger setting.

To investigate whether new strategies could emerge with bigger teams, we conducted additional training with teams of four robots. However, due to limited training time, significant behavioral differences did not appear. We believe that extended training iterations were necessary to discover new cooperative and competitive strategies for this increased team sizes.

VI. CONCLUSIONS AND FUTURE WORK

In the final evaluation, we observe that the agents exhibit predominantly competitive behavior and have not yet fully learned cooperative behavior. These results suggest that, for future work, it may be necessary to readjust the reward function or increase the training time of the model. One suggestion for adjusting the reward function is, for example, to change the balloon-grabbing reward so that it is shared by the entire team rather than being assigned to an individual agent.

We consider the results to be suboptimal because we did not use tags in Unity, which made it more difficult for Unity ML-Agents to recognize patterns. However, we chose this approach because it more realistically reflects how the algorithm would be implemented in a real-world scenario.

The setup for larger team sizes is in place, and was tested. It presented similar results to the two-robot teams. Due to time constraints, it was not possible to train longer this scenario, but we believe that with larger training sessions it would be possible to see better cooperative and competitive results.

Other improvements could be applied to make the game more complete, such as introducing different maps, wrap-around maps or implementing holes.

VII. ACKNOWLEDGMENTS

We would like to thank everyone who contributed to the completion of this work and supported us through the process.

REFERENCES

- [1] V. Silva *et al.*, “Aprendizagem Profunda em Unity com ML-Agents,” in *Minicursos da ERCMAPI 2020*, 1st ed., A. Teles, D. Calçada, and N. Veras, Eds. SBC, Sept. 2020, pp. 146–167. [Online]. Available: <https://sol.sbc.org.br/livros/index.php/sbc/catalog/book/48/221/459-1>. [Accessed: Dec. 28, 2025].
- [2] B. Baker *et al.*, “Emergent Tool Use From Multi-Agent Autocurricula,” in *International Conference on Learning Representations (ICLR)*, 2020. [Online]. Available: <https://arxiv.org/abs/1909.07528>.
- [3] J. Schulman *et al.*, “Proximal Policy Optimization Algorithms,” 2017. [Online]. Available: <http://arxiv.org/abs/1707.09183>. [Accessed: Jan. 14, 2026].
- [4] C. Amato, “An Introduction to Centralized Training for Decentralized Execution in Multi-Agent Reinforcement Learning,” 2024. [Online]. Available: <http://arxiv.org/abs/2409.03052>. [Accessed: Jan. 14, 2026].
- [5] P. Hernandez-Leal, B. Kartal, and M. E. Taylor, “A Survey and Critique of Multiagent Deep Reinforcement Learning,” in *Autonomous Agents and Multi-Agent Systems*, vol. 33, no. 6, pp. 750–797, 2019. [Online]. Available: https://doi.org/10.1007/978-3-030-60990-0_12. [Accessed: Jan. 14, 2026].
- [6] A. Juliani *et al.*, “Unity: A General Platform for Intelligent Agents,” 2018. [Online]. Available: <http://arxiv.org/abs/1809.02627>. [Accessed: Jan. 14, 2026].